Area-Delay-Energy-Efficient Approximate Dividers Based on Piecewise Linear Fitting of Surface

Chaoyuan Wu, Weiwei Shi, *Member, IEEE*, Yida Yuan, Zhuoliang Zou, Zhihong Mo, and Jiangwei He

***Abstract*— In this paper, signed and unsigned approximate dividers based on piecewise linear fitting of surface (PLSAD) are proposed. After extracting the exponent and mantissa of two binary fixed-point operands, the surface representing the quotient of the mantissa is partitioned into sub-regions, which are linearly approximated by rectangular planes. The proposed logic architecture for geometric plane computations relies on simple arithmetic operations of addition and shifting. Additionally, approximate adders reduce circuit complexity and enhance performance, while enabling dynamic configuration of dividers based on accuracy requirements. Fairly compared to recently published approximated dividers based on the same 45nm technology and logic synthesis constrains, the proposed circuits outperform other works in overall aspects. 'As the bit-width of OR-gate additions in Least significant bit (LSB) increases, the mean relative error distance (MRED) gradually grows from 0.82% to 2.63%, while the circuit area, delay and power consumption reduce to 840um^2, 0.9ns and 881uW, respectively. Similarly, the proposed approximate floating-point (FP) 16-bit and 32-bit dividers improve circuit delay and power by over 50% compared to the state-of-the-art designs in the same condition. The proposed dividers perform well in image processing and demonstrate high adaptability in softmax layer of deep neural network and real-time EEG signal recognition applications.**

***Index Terms*— Approximate computing, piecewise linear fitting, divider, energy-efficient, error-tolerant.**

I. Introduction

With the development and popularity of wearable electronic devices and artificial intelligence edge computing, the demand for high-performance and efficient digital electronic systems is rapidly growing. Besides relying on high-cost technology improvement and parallel computational structure, error-tolerant algorithms (Approximate Computing, AC) can be an ideal alternative in image processing, object detection and pattern recognition for better performance, energy consumption and complexity (area). It has been widely applied to digital systems to reduce circuit complexity, power, and delay by sacrificing some accuracy [1], [2], [3], [4], [5].

The majority of the recent approximate computing focuses on adders, compressors and multipliers. The design methods primarily emphasize custom cell design [6], approximations on addition, and accumulation well-organized calculation [7], [8], [9], [10].

Recently, there has been increased attention to approximate design of dividers. In this domain, one particular approach is to approximate the logic expressions of partial adders or subtractorins within the non-restoring and restoring array [11], [12]. The dividers in [13] have only established marginal improvements in hardward performence. Another approach is to enhance algorithm or circuit structure. For example, a high-speed and energy-efficient approximate divider (SEERAD) is proposed in [14]. By rounding off the divisor, this divider saves computational overhead involving only shifting and addition. This simplification leads to a significant reduction in circuit delay while maintaining acceptable precision. However, SEERAD employs lookup tables, resulting in more power consumption and circuit area. Alternatively, TruncApp [15] multiplies the truncated dividend by the approximate inverse of the divisor. TruncApp generally improves power consumption over SEERAD, but its accuracy is rather low due to the direct constant approximation. At the algorithmic level, a promising approximate dividers are based on the logarithmic number system (LNS). These designs use Mitchell's approximate logarithmic conversion to perform division (ALD) [16], leading to significant hardware improvements. However, ALDs are non-configurable and suffer from severe error bias when all error values have the same sign. A hybrid design known as AXHD [17], which combines the restoring array and logarithmic dividers, offers a novel and efficient low-power solution but still leads to major error bias like previous ones. Recently, the field has seen a surge in interest towards approximate dividers utilizing piecewise linear (PWL) techniques. For instance, LPCAD [18] leverages Mitchell's logarithms to convert integer operands into the logarithmic domain. Within this domain, the division operation is approximated by a piecewise constant function, effectively transforming it into a simpler multiplication.

This paper presents a novel hybrid method for division, drawing inspiration from the aforementioned logarithmic conversion and PWL techniques. The proposed approach transforms fixed-point (or integer) division into a more computationally efficient fraction division using logarithmic conversion. Subsequently, the geometric surface representing the quotient space of the fraction division is divided into a set

Manuscript received 3 April 2024; revised 14 June 2024 and 2 July 2024; accepted 3 July 2024. Date of publication 22 July 2024; date of current version 25 October 2024. This work was supported in part by the Natural Science Foundation of Guangdong Province under Grant 2023A1515010761, in part by the Special Innovative Project of Guangdong Universities under Grant 2022KTSCX106, in part by the Shenzhen Science and Technology Program under Grant ZDSYS20220527171402005, in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2021B1101270006, and in part by the National Natural Science Foundation of China under Grant 61974095. This article was recommended by Associate Editor W. Liu. *(Corresponding author: Weiwei Shi.)*

Chaoyuan Wu, Zhuoliang Zou, Zhihong Mo, and Jiangwei He are with the Department of Microelectronics, CEIE, Shenzhen University, Shenzhen 518060, China.

Weiwei Shi is with the State Key Laboratory of Radio Frequency Heterogeneous Integration, Shenzhen University, Shenzhen 518060, China, and also with the Department of Microelectronics, CEIE, Shenzhen University, Shenzhen 518060, China (e-mail: wwshi@szu.edu.cn).

Yida Yuan is with WingSemi Technology (Shanghai), Shanghai 201203, China.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSI.2024.3425538.

Digital Object Identifier 10.1109/TCSI.2024.3425538

of planes, facilitating hardware implementation through the use of shift-and-add circuits. Notably, unlike PWL methods that approximate curves with a limited number of straight lines, this work proposes a Piecewise Linear Fitting of Surface (PLS) approach. PLS approximates the two-dimensional function representing the quotient space with planes, enabling its application in the design of the proposed approximate divider.

The description of the proposed approximation method and the hardware architecture is provided in Section II. Section III presents the error analysis, synthesis results and comparisons. In Section IV, some practical applications of our design are shown. Section V concludes the paper.

II. Proposed Approximate Divider Design

This section details the derivation of the principles behind the proposed PLS-based approximate divider, as well as the corresponding implementation of hardware circuit structure.

A. Derivation Analysis and Proposed Approximation of Division

First of all, we can convert the input operands A and B of the divider as follows, both of which are non-zero positive integers.

$$A = 2^{e_1}(1 + f_1) \tag{1}$$

$$B = 2^{e_2}(1 + f_2) \tag{2}$$

In the expression above, e_1 and e_2 are the exponents of A and B in float-point format which can represent the position of the leading-one bit in binary. f_1 and f_2 are the fraction part of A and B within the range of [0,1). Thus, the result of the division of two operands, Q can be computed using the following equation.

$$Q = \frac{A}{B} = \frac{2^{e_1}(1 + f_1)}{2^{e_2}(1 + f_2)} = 2^{e_1 - e_2}\frac{1 + f_1}{1 + f_2} \tag{3}$$

$$y = \frac{1 + f_1}{1 + f_2} \tag{4}$$

The range for the quotient of $(1 + f_1)/(1 + f_2)$ is [0.5,2), and in Eq. 4, we replace it by y which represents a surface. Applying calculus, we may employ a plane to approximate the area of the surface with a small region σ. The expression of this plane can be written as follows

$$\hat{y} = \left(\frac{1 + f_1}{1 + f_2}\right)_{approx} = k_1 f_1 + k_2 f_2 + C \tag{5}$$

$\hat{y}$ is the approximate value of y for the region σ, k_1 and k_2 are the coefficients of the variables f_1 and f_2, respectively, and C is a constant offset. It is not feasible to fit the entire surface with an infinite number of planes due to the significant resource overhead on the hardware circuit. Nevertheless, we can minimize error by using a finite number of planes as a substitute for the surface. Fig. 1(a) shows plotted y in MATLAB. We rotated the coordinate axis to view y-f_1, revealing a quadrilateral plane projection as depicted in Fig. 1(b). We calculated the primary and secondary partial derivatives of f_1 for y, yielding the following expressions

$$\frac{\partial y}{\partial f_1} = \frac{1}{1 + f_2} \tag{6}$$

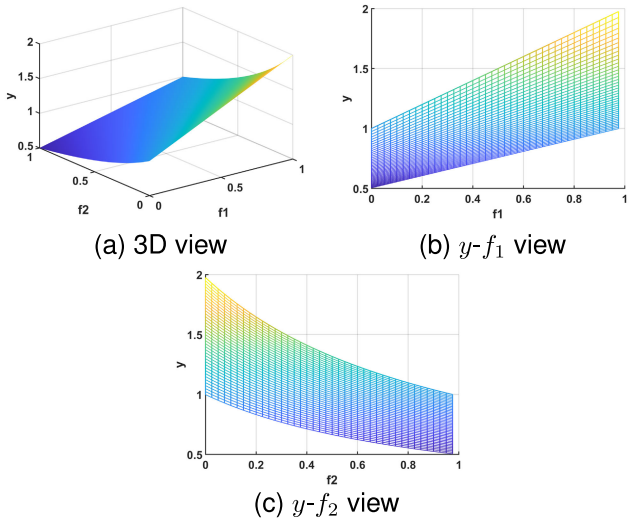

Fig. 1. The surface of $(1 + f_1)/(1 + f_2)$.

$$\frac{\partial^2 y}{\partial {f_1}^2} = 0 \tag{7}$$

Eqs. 6 and 7 indicate that given a fixed value of f_2(named a) and any value of f_1 in the range [0,1), the shadow of y projected onto the view y-f_1 is a line with curvature 0 and perpendicular to the f_2 axis, with a slope of $1/(1+a)$. To fit a surface, we can utilize an infinite number of these lines or planes with a width of small Δf_2 and length 1. Similarly, taking the partial derivative of f_2, y gets

$$\frac{\partial y}{\partial f_2} = -\frac{1 + f_1}{(1 + f_2)^2} \tag{8}$$

$$\frac{\partial^2 y}{\partial {f_2}^2} = \frac{2\,(1 + f_1)}{(1 + f_2)^3} \tag{9}$$

Eq. 9 is a function with respect to f_1 and f_2. Even though f_1 takes a fixed value, the projection of y onto the view y-f_2(as shown in Fig. 1(c)) exhibits a variable curvature across different values of f_2. Therefore, we adopted a strategy of dividing the f_1-f_2 plane into several regions using a line perpendicular to the f_2 axis and derived PLS functions for each region. While a smaller increment of Δf_2 leads to a better surface fitting, there exists a hardware-accurate trade-off to be taken into account. Having too many or too few regions might strain circuit resources or increase error rates, we divided the [0,1) range of f_2 into eight equal-sized regions. The k_1,k_2 can then be solved for next.

$$k = \int_{\sigma_i}\int \frac{\partial y}{\partial f} d\sigma_i / \sigma_i \tag{10}$$

Assuming that the region σ_i is defined by $0 \le f_1 < 1$ and $i/8 \le f_2 < (i + 1)/8$, k_1 and k_2 of $\hat{y}$ can be approximated by the mean value of the integrals of $\partial y/\partial f$ with respect to f_1 and f_2 over σ_i, respectively. By applying the integral mean theorem, we can compute the definite integral of the partial derivatives of y over the σ_i, divide it by the area of σ_i, and obtain the mean value as shown in Eq.10. Combining Eq.6, 8 and 10, we can obtain expressions for the calculation of the

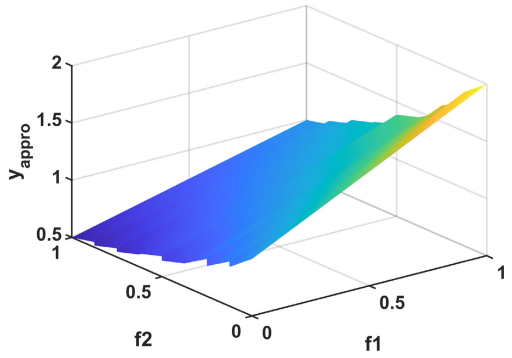

Fig. 2. The 3D view of y_{appro}.

k_1 and k_2, as shown in Eq. 11 and Eq. 12.

$$k_1 = \frac{\int\int \frac{\partial y}{\partial f_1} d\sigma_i}{\sigma_i} = \frac{\int_{\frac{i}{8}}^{\frac{i+1}{8}} \frac{1}{1+f_2} df_2}{\frac{i+1}{8} - \frac{i}{8}} = 8\ln\frac{9+i}{8+i} \tag{11}$$

$$k_2 = \frac{\int\int \frac{\partial y}{\partial f_2} d\sigma_i}{\sigma_i} = -\frac{3}{2(1+\frac{i}{8})(1+\frac{i+1}{8})} \tag{12}$$

To simplify the hardware calculations, the values of k_1 and k_2 are approximated to $(2^{-m}+2^{-n})$, so that we can multiply k_1, k_2 through shifting and adding operations in the circuit. Finally, the constant offset C is adjusted. To find a suitable C value, we sampled 100×100 points equally in each region and calculated the minimum Mean Relative Error Distance (MRED, in Eq. 13) between the surface and the linear approximation plane. Similarly, C is rounded to $(2^{-m}+2^{-n})$.

$$MRED = \frac{1}{N}\sum_{1}^{N} \left|\frac{Accurate - Approximate}{Accurate}\right| \tag{13}$$

N is the number of sample points. The final plane functions for the y_{approx} of the eight regions are obtained as follows. And Fig. 2 shows the coordinate diagram after piecewise linear fitting for the surface y.

$$y = \begin{cases} f_1 - \left(1+\frac{1}{4}\right)f_2 + 1, & f_2 \in \left[0, \frac{1}{8}\right) \\ \left(1-\frac{1}{8}\right)f_1 - f_2 + \frac{65}{64}, & f_2 \in \left[\frac{1}{8}, \frac{2}{8}\right) \\ \left(1-\frac{1}{4}\right)f_1 - f_2 + \frac{69}{64}, & f_2 \in \left[\frac{2}{8}, \frac{3}{8}\right) \\ \left(1-\frac{1}{4}\right)f_1 - \frac{1}{2}f_2 + \frac{57}{64}, & f_2 \in \left[\frac{3}{8}, \frac{4}{8}\right) \\ \left(\frac{1}{2}+\frac{1}{8}\right)f_1 - \frac{1}{2}f_2 + \frac{119}{128}, & f_2 \in \left[\frac{4}{8}, \frac{5}{8}\right) \\ \left(\frac{1}{2}+\frac{1}{8}\right)f_1 - \frac{1}{2}f_2 + \frac{59}{64}, & f_2 \in \left[\frac{5}{8}, \frac{6}{8}\right) \\ \left(\frac{1}{2}+\frac{1}{16}\right)f_1 - \frac{1}{2}f_2 + \frac{61}{64}, & f_2 \in \left[\frac{6}{8}, \frac{7}{8}\right) \\ \frac{1}{2}f_1 - \left(\frac{1}{2}-\frac{1}{8}\right)f_2 + \frac{7}{8}, & f_2 \in \left[\frac{7}{8}, 1\right), \end{cases} \quad f_1 \in \left[0, 1\right) \tag{14}$$

TABLE I
THE VALUE OF k_1, k_2 IN REGIONS

Region	k_1	k_2	Region	k_1	k_2
σ_0	0.9423	-1.332	σ_4	0.6403	-0.615
σ_1	0.8429	-1.066	σ_5	0.5929	-0.5272
σ_2	0.7625	-0.872	σ_6	0.5519	-0.4569
σ_3	0.6961	-0.7267	σ_7	0.5163	-0.3998

TABLE II
THE SUMMARY OF THE THREE CASES OF PLS PLANES

Plane Number	4	8	16
Number of Addends	4	4	6
MRED(%)	1.30	0.82	0.37
Area(um^2)	1018	1072	1506
Delay(ns)	1.02	1.06	1.34
Power(mW)	1.51	1.67	2.35

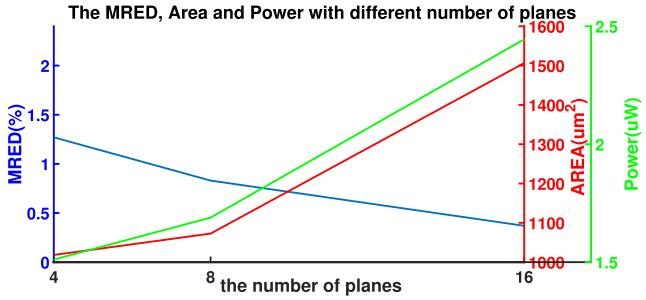

Fig. 3. The MRED, area and power with different number of planes.

The analysis given here explains why eight planes were chosen instead of four or sixteen.

When implementing the algorithm on circuits, we will use a multiplexer (MUX) to make the selection of each plane. To accommodate for binary system characteristics, we divide the surface evenly into planes that are multiples of the power of 2. It is easier to control the bit width of the MUX address lines and ensures as much consistency as possible in the hardware circuitry and software algorithms.

According to Eq. 14, the PLS algorithm can achieve multiplication of constant coefficients through multiple addition operations. PLS for 4 and 8 planes is both converted to the addition of 4 addends, while 16 planes require 6 addends due to finer dividing of $f2$ and more precise $k1$ and $k2$ values.

Table II evaluates the MRED and circuit performance of approximation 16-bit dividers divided into 4, 8, and 16 planes. Constrained by the same RTL synthesis settings specified in Section III-B, the design explores the trade-off between accuracy and hardware cost. As the number of planes employed in the PLS method increases, the MRED of the algorithmic model demonstrably decreases. However, this improvement in accuracy comes at the cost of higher resource consumption within the synthesized circuit. In the 16-plane design, MRED is the smallest, but the circuit resource consumption and the power are the highest. The 4-plane design has the highest MRED (1.3%), however there is no significant reduction in circuit area compared to the 8-plane one as shown in Fig. 3. Since both of them involve 4 additions, the 4-plane design has slightly fewer MUXs than the 8-plane design. The MRED

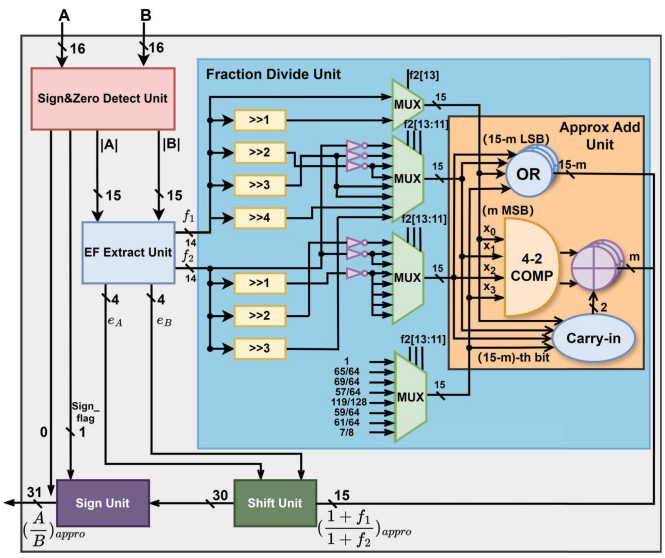

Fig. 4. Hardward architecture of the proposed design.

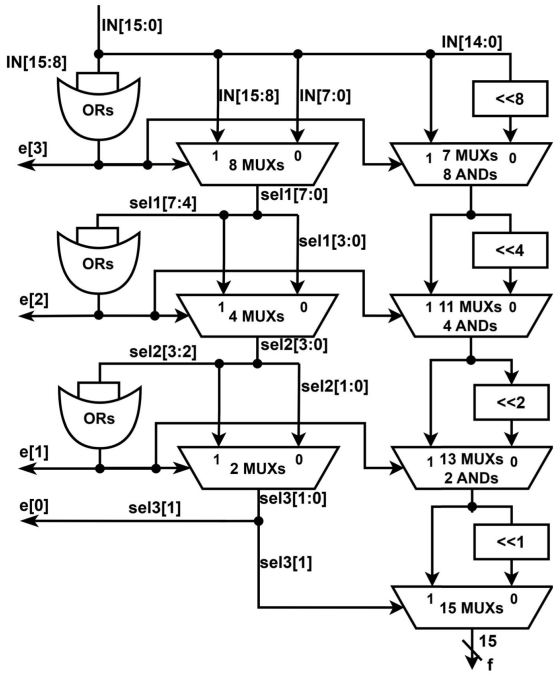

Fig. 5. The hardware structure of EF extract Unit for 16-bit input.

TABLE III
HARDWARE COMPARISON OF EXPONENT-FRACTION EXTRACTION DESIGNS

	area(um^2)	delay(ns)	power(uW)
ours	176	0.41	166
[19]	225	0.52	229
[20]	347	0.59	241
[21]	206	0.46	165

of 0.82% for the 8-plane design is acceptable enough in most error-tolerant applications. Moreover, the circuit area and delay are reduced by 29% and 20.8%, respectively, compared to the 16-plane design. To balance tolerable computational error rates with better circuit performance, we chose the 8-plane design.

B. Logic Architecture Design and Hardware Optimization

Taking signed 16-by-16 division as an example, a block diagram of the proposed approximate divider structure is depicted in Fig. 4. In this work, the input of the divider is assumed to be a 16-bit fixed-point number in 2's complement format. To handle zero inputs, a Sign&Zero Detect Unit is employed to identify if the input contains any zeros. In this case, the output of the divider is set to zero. Otherwise, the unit detects the sign bit of the dividend or divisor to determine whether a positive value conversion for dividend or divisor is necessary. Meanwhile, the unit transmits the sign flag through an XOR gate to the Sign Unit.

The absolute values of operands A and B, denoted $|A|, |B|$ are processed by the EF Extract Unit. This unit isolates the exponent e (the position of leading-one bit) and the fraction f (lying in the range of [0,1)), which are 4-bit and 14-bit respectively. In [19] and [20], the Leading One Detectors (LODs), the Encoders (ENCs) and the left-shifter are employed for this extraction. It must be noted that the LOD module produces a one-hot code of the leading-one bit, which needs to be encoded to a binary number by ENCs and then left-shift the input to get the fractional part. This paper proposes a more efficient approach that eliminates the need for these additional steps. lgorithm 1 details a novel approach for efficient exponent and fraction extraction applicable to n-bit input operands, where n is a power of 2. The algorithm iteratively divides the input into two halves and selects the appropriate half based on the presence of leading one bits. This selection process effectively determines the position of leading one bit (exponent, e) and identifies the fractional portion (f) requiring a fixed-length left shift. The algorithm repeats for $log_2(n)$ iterations, culminating in the desired exponent and fraction values. For illustrative purposes, consider a 16-bit unsigned input (detailed in Fig. 5). The EF Extract Unit leverages OR gates to detect the presence of leading one bit within the upper half of the input. This detection governs the selection of either the high or low half-bits by MUX circuits, along with the determination of a fixed left shift for the chosen portion. Notably, the output of the OR gates directly contributes to the individual bits of the exponent (e). The efficiency of this approach is demonstrated by the EF Extract Unit requiring only four executions of OR-MUX to produce a valid fraction (f) at the final MUX circuit.

For comparative evaluation, we implemented circuits employing the LOD methods presented in [19], [20], and [21], ENCs and left-shifter to extract the exponent and fraction from a 16-bit integer or fixed-point input. Notably, the LOD approach in [19] approximates the lowest 8 bit as 2 4-bit inexact LOD blocks, which have been reproduced following its original description. The leading-one position detector (LOPD) of [21] can directly output the binary number of the leading-one position without the ENCs. Table III summarizes the synthesized circuit characteristics under identical conditions. Our proposed method demonstrably achieves the smallest circuit area and the shortest critical path, surpassing the best performing reference [21] by 15% in area and 11% in delay. This improvement can be attributed to the streamlined design of our circuit, which eliminates the priority encoder and shifter. In simulation, the EF Extract Unit consumes less than 20% power of the proposed divider, while the percentages of the Fraction Divide Unit, the Shift Unit and other units/output buffers are around 39.0%, 29.6% and 11.5% respectively.

Algorithm 1 EF Extracting.

$f = input[n-2:0]$
$select = input$
for $i = 1 : log_2 n$ **do**
  if $select[\frac{n}{i} - 1 : \frac{n}{2i}]! = 0$ **then**
    $select = select[\frac{n}{i} - 1 : \frac{n}{2i}]$;
    $e[log_2 n - i] = 1$;
    $f = f << (2^{log_2 n - i})$;
  else
    $select = select[\frac{n}{2i} - 1 : 0]$;
    $e[log_2 n - i] = 0$;
    $f = f$;
  end if
end for

The Fraction Divide Unit calculates the y_{approx}, and its internal circuit block diagram is shown in Fig. 4. After obtaining f_1 and f_2, the appropriate values for k_1, k_2 and C can be determined based on the value of f_2[13:11]. If k_1 or k_2 is negative, calculating it with 1's complement can reduce the delay and resources of adding 1. With C, we get 4 addends, x_0, x_1, x_2, x_3, which will be summed in Approx Add Unit. The bit width of $x_3(C)$ is 15, where $x_3[14]$ represents integer bit and $x_3[13:0]$ are fractional bits. x_0, x_1, x_2 are also expanded by one bit at the most significant bit (MSB). Since the quotient of $(1+f_1)/(1+f_2)$ is less than 2, the sum of the adder should also be a 15-bit number. In this study, a 4-2 compressor is used instead of full adder array of 4 inputs. The conventional 4-2 compressor, comprised of two full adders connected in series, produces sum or carry-out signals with a delay of 4Δ(assuming Δ is the delay of a XOR gate or a carry generator). Therefore, a flattened 4-2 compressor [22] can reduce 1Δ delay more than the conventional one. Consequently, multiple 4-bit CLA cascades are used to compute the sum of two outputs from the 4-2 compressor to get the quotient of $(1+f_1)/(1+f_2)$.

In pursuit of reducing circuit area and mitigating the long carry-in delay of multi-bit adders, this work employs an approximate adder techniques. The operands of adders are fixed-point representations of fraction, whose least significant bits (LSBs) hold minimal weight. Consequently, these LSBs can be truncated or approximated with minimal impact on the overall accuracy. However, truncation would introduce a negative bias, so we adopt an approximate method named Lower Part OR Adder (LOA) [1], [23] that can have an almost unbiased and symmetric output error range between signed or unsigned addition, and expand its 2 inputs to 4 inputs. In Approx Add Unit, the exact adding are executed in the MSB m-bit of four addends, while the rest bits of addends use approximate adders consists of OR-gates as shown in Fig. 6. To minimise the error caused by the OR-gate addition, the carry signal cin_1 passed to the $(14-m+1)^{th}$ bit will be set high if there are two or more 1s at the $(14-m)^{th}$ bit, and the second carry signal cin_2 will be set high if there are four 1s.

In the circuit, cin_1 can be placed at the least significant bit of the output $carry$ from the 4-2 compressor, before using CLA to add the sum and $carry$. The cin_2 can serve as the carry-in signal of CLA. The dot diagram of the exact 4-2 compressing adder and the approximate LOA are shown in Fig. 7. The grey circles in Fig. 8(b) represent the part that uses the OR-gate approximate addition. cin_1 and cin_2 are not directly connected to the carry-in signal of the 4-2 compressor, which prevents blocking and allows the or-tree adding and 4-2 compressing to be processed in parallel. Decreased bit width of exact addition (m) reduces hardware resources, but also increases inaccuracy.

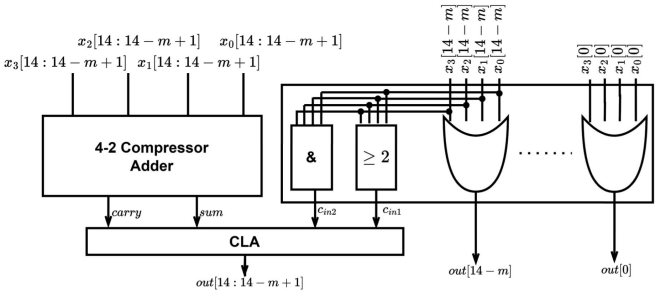

Fig. 6. The structure of LOA with m exact bits.

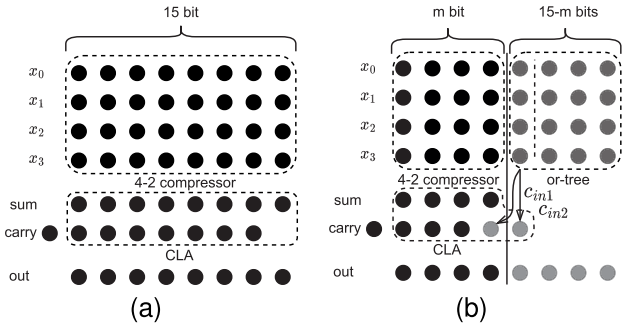

Fig. 7. Dot diagram of the (a) exact and (b) approximate adding step.

The output of the Fraction Divide Unit is shifted left or right by the Shift Unit based on the value of $e_1 - e_2$. If $e_1 - e_2$ is negative, then the output is shifted to the right by $|e_1 - e_2|$ bit, otherwise it is shifted to the left by $|e_1 - e_2|$ bit. Finally, the sign unit determines the sign of the output based on the sign flag. The $(\frac{A}{B})_{approx}$ yields a 31-bit result of PLSAD, where the LSB 14 bits are fractional and the MSB 15 bits are integer bits in addition to the sign bit. In the case of unsigned dividing, the Sign Detect and Sign Unit can be eliminated from the design. The divider produces a 31-bit output with 16 integer bits at MSB and 15 fractional bits at LSB.

Compared to previous works, the features of the proposed divider design are as follows.

1) A novel hybrid method with logarithmic conversion and piecewise surface calculation is proposed. We mathematically formulate a process for surface dividing of the quotient space arising from the logarithmic conversion of mantissa values. In contrast to simple curve-fitting, we fit a two-dimensional surface function by combining the linear representations of the two variables, which converts the complex computation into shift-and-add circuits that are more efficient for hardware implementation.
2) The trade-off between hardware cost and accuracy is explored by finding a suitable number of planes and

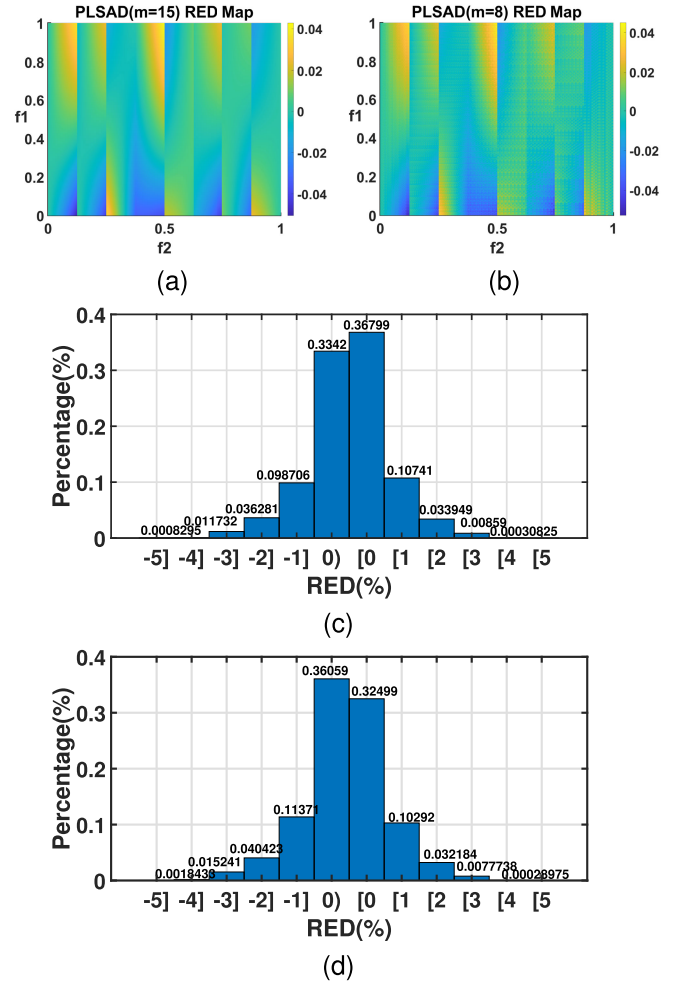

Fig. 8. The RED Map and the RED Distribution of 16-by-16 PLSAD with (a)(c) m = 15 and (b)(d) m = 8.

approximate the coefficients of variable to the power of 2 to reduce the number of adders required. In addition to introducing the LOA technique extended to 4-input to further reduce circuit complexity and shorten the critical path for an hardware-efficient approximate divider with configurable accuracy. Both fixed-point and floating-point divider structures are also presented in this paper.

3) In terms of accuracy and circuit performance, the proposed divider is compared with state-of-the-art approximate divider in terms of MRED, ADP and EDP metrics, including the latest AHXD, FPDME, LPCAD and so on. The results show that the MRED of the proposed divider is minimal and is in the Pareto Frontier when both ADP and EDP are considered.
4) The proposed approximate divider is further evaluated in two image processing, softmax function of deep neural network, and real-time emotion recognition applications to validate the effectiveness of the proposed design.

III. Error Analysis and Implementation Evaluation

In this section we present simulation results for the analysis of error metrics in Relative Error Distance (RED), MRED, Normalised Mean Error Distance (NMED) by the maximum exact output, Error Bias (EB), and overall results for circuit synthesis. Additionally, we compare our proposed design with several advanced approximate dividers.

A. Error Analysis

In contrast to the exact divider, we introduce errors primarily during the calculation of the fraction division. Thus, the RED without LOA can be expressed as Eq. 15.

$$RED = \frac{\left(\frac{A}{B}\right)_{appro} - \frac{A}{B}}{\frac{A}{B}} = \frac{\left(\frac{1+f_1}{1+f_2}\right)_{appro} - \frac{1+f_1}{1+f_2}}{\frac{1+f_1}{1+f_2}} = \frac{(k_1 f_1 + k_2 f_2 + C) - \frac{1+f_1}{1+f_2}}{\frac{1+f_1}{1+f_2}} \tag{15}$$

$$EB = \frac{1}{N}\sum_{1}^{N} \frac{(k_1 f_1 + k_2 f_2 + C) - \frac{1+f_1}{1+f_2}}{\frac{1+f_1}{1+f_2}} \tag{16}$$

According to Eq. 15, the $\left[(k_1 f_1 + k_2 f_2 + b) - \frac{1+f_1}{1+f_2}\right]$ can be both positive or negative. Therefore, the errors in approximate division are two-sided and can be eliminated from each other in the cumulative operation. subsequently we introduce the EB, expressed as Eq. 16, that can measure the degree of error introduced in the iterative calculation. Smaller EB accumulates fewer errors. The LOA replaces the exact adder, lowering hardware costs. However, it causes a minor mistake in the addition result, especially for small m. Similarly, RED with m-bit exact adder can be calculated using Eq. 17.

$$RED_m = \frac{\lfloor k_1 f_1 + k_2 f_2 + C \rfloor_m - \frac{1+f_1}{1+f_2}}{\frac{1+f_1}{1+f_2}} \tag{17}$$

To evaluate the MRED, NMED, and EB of the proposed 16-by-16 and 8-by-8 dividers, we employ an exhaustive evaluation strategy. This approach considers all possible input combinations for operands f_1 and f_2. For each combination, the error between the approximate divider output and the exact result $(1 + f_1)/(1 + f_2)$ is calculated. Fig. 8(a)(b) shows the RED distribution on the f_1-f_2 plane for 16-by-16 PLSAD, The color value mapping is generally symmetric about the line RED=0, leading EB to be closer to 0. For the PLSAD with m=8 in Fig. 8(b), the RED plot appears less smooth due to the use of approximate OR-gate addition instead of exact addition in the LSBs. On the contrary, as shown in Fig. 8(c) and (d), both of them have small and concentrated RED distribution.

Table IV shows the simulation results for the error metrics, NMED, MRED, and EB, for both unsigned 16-by-16 and 8-by-8 PLSADs, which were evaluated by using exhaustive 65536 × 65536 and 256 × 256 input cases, respectively. The line graph in Fig. 9 illustrates the relationship between m and statistical errors of MRED, NMED, and |EB| for the 16-by-16 PLSAD. As the value of m decreases, the bit width of LOA increases, introducing more errors for divider. In the 16-by-16 design, MRED barely changes at $m \geq 8$ because the bit weight of the 8 bits at the LSB is only a small part of it compared to the whole design. When m<8, the proportion of bit weights in LOA increases, leading to a more rapid error growth. Thus, a better trade-off between error and hardware performance can

TABLE IV

MRED(%), NMED(10^{-6}) AND EB(%) OF UNSIGNED 8-BY-8 AND 16-BY-16 PLSADS

8-by-8				16-by-16			
m	MRED (%)	NMED (10^{-6})	EB (%)	m	MRED (%)	NMED (10^{-6})	EB (%)
2	8.77	599.4	-5.54	2	8.41	6.41	-4.94
3	5.39	343.5	-4.06	4	2.63	2.06	-1.83
4	3.25	183.8	-2.50	6	0.97	0.74	-0.19
5	1.98	100.9	-1.30	8	0.84	0.63	-0.049
6	1.65	82.3	-1.11	10	0.83	0.62	-0.011
7	1.59	78.2	-0.99	12	0.83	0.62	-0.002
				14	0.83	0.62	0.002

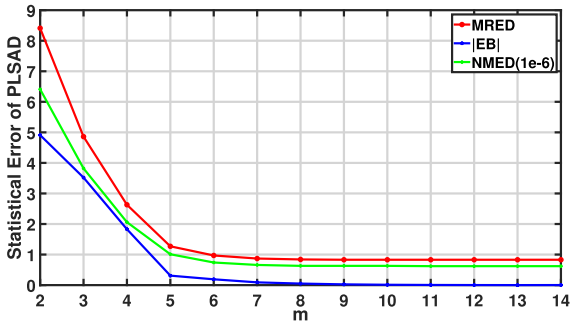

Fig. 9. The MRED, NMED and EB of unsigned 16-by-16 PLSAD with different m.

be achieved by using a suitable m value, which will be further investigated in the following subsections.

B. Hardware Evaluation and Comparison

The proposed approximate divider and the state-of-the-art dividers were implemented and reproduced in Verilog HDL and synthesized by Synopsys Design Compiler. For fair comparisons, all the designs were synthesized based on the NanGate 45 nm Open Cell Library [24], with the same constraints of delay, power, area and capacitive load etc. After synthesis, dividers are mapped to gate-level netlists, which are later used to run static timing analysis (STA) and PTPX to measure the circuit delay and average power consumption. In order to provide a comprehensive evaluation of the divider, we have established three extra design metrics: Energy-Delay Product (EDP), Area-Delay Product (ADP), and Power-Delay-Area Product (PPA). These metrics provide a combined view of circuit efficiency, considering both accuracy and hardware cost. Table V shows the area, power, delay, EDP, ADP, and PPA of the unsigned and signed 16-by-16 PLSAD with different m value, respectively.

The proposed PLSAD architecture leverages variable-precision exact adders controlled by the parameter m (m bits). This design balances hardware complexity (area, power, and delay) with accuracy (MRED). We observed minimal MRED variation in the PLSAD divider for $m \geq 8$. Therefore, PLSAD(8) offers the optimal trade-off, achieving the lowest MRED while maximizing hardware efficiency. To facilitate a comprehensive comparison, we implemented PLSAD variants with m = 8, 6, and 4. As mentioned, we assume 16-bit dividends and 8-bit divisors of PLSAD for a fair comparison with resroting array dividers and 16-bit dividends and 16-bit divisors for a fair comparison with other logarithmic dividers, respectively. PLSAD is not restricted to any specific dividend-divisor bit ratio.

TABLE V

EXPERIMENTAL RESULTS OF THE PROPOSED UNSIGNED AND SIGNED 16-BY-16 DIVIDER

PLSAD	m	Area (um^2)	Delay (ns)	Power (uW)	EDP	ADP	PPA ($\times 10^3$)
Unsigned	4	840	0.90	881	713	756	666
	6	908	0.93	1022	884	845	863
	8	936	0.96	1179	1087	899	1059
	10	979	1.00	1347	1347	979	1319
	12	1037	1.03	1460	1549	1068	1559
	14	1063	1.06	1597	1795	1136	1799
Signed	4	973	0.96	1254	1156	934	1171
	6	1044	1.03	1490	1581	1075	1602
	8	1104	1.11	1732	2134	1225	2122
	10	1167	1.12	1812	2274	1309	2368
	12	1233	1.14	2032	2642	1405	2856
	14	1271	1.17	2097	2872	1486	3118

Table VI lists the logic synthesis results of proposed designs as well as those of AXHD [17], AXDr3 [11], AXRD [26], TruncApp [15], SAADI [27] and LEDAD [28]. EDP, PDP and MRED are given in the three columns on the right. It is obvious that PLSAD exhibits the shortest critical path among all the dividers and also offers the smallest EDP and ADP when considering similar MREDs. As shown in Fig. 10, it reveals the relationship between MRED and EDP of the 16-by-8 approximate dividers. The location of proposed design are at the left-bottom corner, which implys that the energy consumption and circuit responsiveness of PLASD are at the Pareto level for the same accuracy requirement. However, for 16-by-8 dividers, ADM1(8) and AXRD-M2(4) achieve the smallest circuit area, but albeit at the expense of increased delay due to their complex array structures. E-AXHD(14) offers the small area through a hybrid restoring array and logarithmic divider architecture, further reducing complexity by eliminating the divisor operand. AXDr3 using the TR scheme generally has the smallest MRED but similarly the delay of its complex array circuit is the longest here. For 16-by-16 dividers, TruncApp designs achieve minimal area and delay by truncating the least significant bits (LSBs) of both operands and converting the divisor's inverse to a constant multiplication with the dividend. However, this simplification comes at the expense of significantly higher MRED. SAADI_EC(8) utilizes a Taylor series expansion for division, achieving the lowest MRED but incurring the highest delay (10.15 ns) due to its seven-iteration process. Similarly, LEAD achieves low MRED through an exponent series expansion in the logarithmic division; however, this method suffers from high area and long circuit paths due to the use of multipliers.

To sum up, the proposed divider outperforms the other approximate dividers considered in this work in terms of hardware cost when the same MRED is needed.

C. Evaluation and Comparison On FPGA

Field Programmable Gate Array(FPGA) are semiconductor devices consisting of a matrix of Configurable Logic

TABLE VI
COMPREHENSIVE COMPARISONS OF UNSIGNED 16-BIT APPROXIMATE DIVIDER DESIGNS

Design	Area (um^2)	Delay (ns)	Power (uW)	EDP	ADP	MRED (%)
PLSAD(4)*	455	**0.89**	**542**	**0.64**	**0.64**	2.63
PLSAD(6)*	498	**0.91**	680	**0.84**	**0.86**	0.97
PLSAD(8)*	567	**0.93**	774	**1.00**	**1.00**	0.84
PLSAD(4)	840	**0.90**	881	**0.65**	**0.84**	2.63
PLSAD(6)	908	**0.93**	1022	**0.81**	**0.94**	0.97
PLSAD(8)	936	**0.96**	1180	**1.00**	**1.00**	0.84
AXHD(8)* [17]	988	3.5	1109	20.27	6.55	**0.34**
AXHD(12)* [17]	715	2.06	654	4.14	2.79	1.41
E-AXHD(10)* [17]	864	2.71	743	8.14	4.44	1.83
E-AXHD(14)* [17]	518	1.28	**575**	1.41	1.26	2.03
AXDr3_TR(8)* [11]	562	2.84	975	11.74	3.03	**0.29**
AXDr3_TR(12)* [11]	499	2.79	817	9.49	2.64	2.86
ADM1(6)* [25]	525	2.41	1215	10.53	2.40	0.50
ADM1(8)* [25]	**387**	1.98	878	5.14	1.45	1.91
ADM2(2)* [25]	724	2.54	1333	12.84	3.49	1.18
ADM2(4)* [25]	539	1.84	833	4.21	1.88	3.98
AXRD-M2(2)* [26]	503	2.25	955	7.21	2.14	1.07
AXRD-M2(3)* [26]	411	1.89	708	3.78	1.47	2.35
AXRD-M2(4)* [26]	**352**	1.69	654	2.79	1.13	4.49
TruncApp(3) [15]	855	1.06	838	0.87	1.01	9.80
TruncApp(4) [15]	933	1.16	1039	1.29	1.20	4.22
TruncApp_AM(5) [15]	963	1.17	1070	1.35	1.25	3.76
SAADI_EC(8) [27]	1927	1.45 (10.15)	2570	4.96 (242.86)	3.11 (21.77)	5.77 (**0.37**)
LEAD [28]	2151	1.96	4816	16.97	4.69	2.18

* denotes 16-by-8 dividers, others are 16-by-16 dividers.

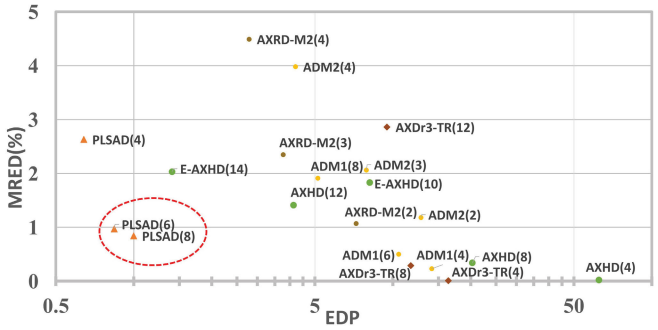

Fig. 10. Joint EDP and MRED comparison of 16-by-8 PLSAD and other state-of-art dividers.

Blocks(CLBs) connected by programmable interconnects. FPGA can be reprogrammed to implement the logic circuits required by the user.

To examine the applicability of the proposed approximate computation method on FPGA, the XA7Z020 was selected as the FPGA device and we synthesized the PLSAD design through the Xilinx Vivado tool to compare it with the Divider Generator IP in terms of logic resource utilization(mainly Look-Up Table, LUT) and worst path delay.

Before using the Divider Generator IP, the Output Remainder Type was configured as Fractional and fractional width is taken as half of the output width, since our design also retains the fractional part. For example, for a signed 16-by-16 divider, the output format of IP should be 1 signed bit, 16 integer bits, and 15 fractional bits. On the configuration table of IP, the Algorithm Type is Radix2, the operand sign option is Signed, and the Number of Latency is 0 that will only generate the combinational logic circuit without flip-flop. After

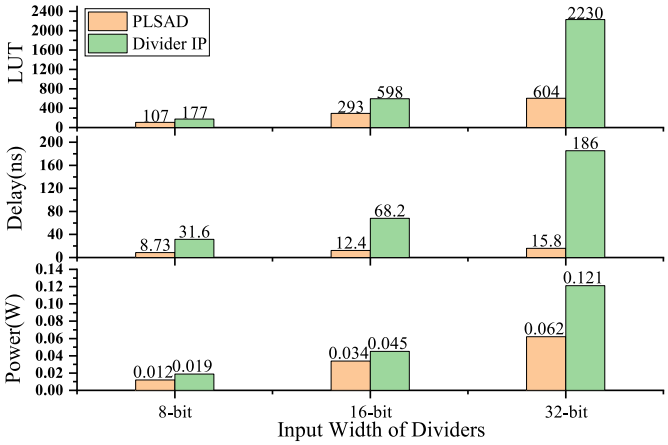

Fig. 11. The Comparison of FPGA implementation in LUTs, delay and power of dividers with difference input width.

completing the configuration, flip-flops are added to the input ports and output ports of dividers to constrain the timing of the combinational circuit. And the clock frequency is set to 100MHz, to estimate the LUT utilization, worst path delay, and on-chip dynamic power on FPGA.

Fig. 11 shows the number of LUTs, worst path delay, and dynamic power for three different input width of dividers. The m of 16-bit and 32-bit PLSAD is 8. The PLS method uses shift-addition operations to achieve complex non-linear calculations, so the LUT resources used in the FPGA are proportional to the bit-width of the inputs. Whereas for Xlinix's Divider IP, the number of LUT tends to grow exponentially with increasing input width. Moreover, the power consumption and the worst path delay of PLSAD are significantly better than those of Divider IP.

The proposed design surpasses Divider IP in terms of resource efficiency, power consumption, and circuit latency for divisions with more input widths.

D. Floating Point Divider Implementation and Comparison

The proposed approximate method is also suitable for floating-point dividers. In the signed 16-bit floating point divider, the exponent and mantissa are directly obtained from the binary of input, so only approximate division of the mantissa is required, as shown in Fig. 12. The circuits of DIV module are the same to the Fraction Divide Unit of Fig. 4. For the standard half-precision(16-bit) representation specified in IEEE 754 standard, they are 5 and 10 bits, respectively, denoted as FP (1, 5, 10).

LPCAD [18], FPDME [29] and FPAD [30] are the state-of-the-art FP dividers. To make a comparison with proposed designs, we have reproduced these designs and synthesized them based on the NanGate 45 nm Open Cell Library [24] using the same constraint conditions. In addition to the signed 16-bit FP divider, we have compared the 32-bit FP version, which is denoted as FP (1, 8, 23). The mantissa of inputs is truncated to 10 bits in our designs.

Table VII summarizes the comparative performance of the proposed 16-bit and 32-bit floating-point dividers against state-of-art established designs. The proposed PLSAD architecture demonstrates a favorable trade-off between MRED and EDP.

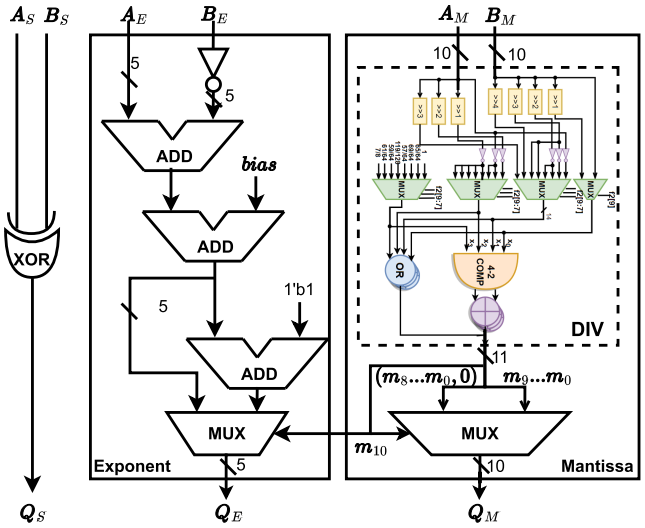

Fig. 12. The proposed PLSAD hardware architecture for FP numbers.

TABLE VII
COMPARISONS OF 16-BIT FP DIVIDERS AND 32-BIT FP DIVIDERS

16-bit FP Dividers						
Design	Area (um^2)	Delay (ns)	Power (uW)	EDP	APD	MRED (%)
PLSAD(4)	**250**	**0.58**	292	**0.41**	**0.59**	2.70
PLSAD(6)	309	**0.63**	381	**0.71**	**0.81**	0.99
PLSAD(8)	364	**0.67**	529	1.00	1.00	0.84
LPCAD(2,10) [18]	365	0.80	718	1.93	1.20	1.30
LPCAD(3,10) [18]	446	0.83	833	2.41	1.52	**0.75**
$FPDME_4$(7,1,3,no trunc.) [29]	343	0.66	477	**0.87**	**0.93**	1.41
$FPDME_8$(7,1,4,no trunc.) [29]	387	0.73	746	1.67	1.16	0.77
FPAD_L4A2 [30]	329	0.60	437	**0.65**	**0.81**	4.61
FPAD_L4A3 [30]	561	0.77	689	1.72	1.77	2.88
32-bit FP Dividers						
Design	Area (um^2)	Delay (ns)	Power (uW)	EDP	ADP	MRED (%)
PLSAD(4)	301	**0.59**	365	**0.47**	**0.63**	7.10
PLSAD(6)	356	**0.64**	450	**0.68**	**0.81**	1.21
PLSAD(8)	418	**0.67**	601	1.00	1.00	0.85
LPCAD(3,4) [18]	**273**	0.63	427	**0.63**	**0.61**	2.54
LPCAD(3,8) [18]	390	0.79	727	1.69	1.10	0.77
LPCAD(3,12) [18]	534	0.87	1077	3.03	1.66	**0.74**
$FPDME_4$(7,1,3,no trunc.) [29]	873	0.73	1296	2.56	2.27	1.41
$FPDME_4$(7,1,3,nt=17) [29]	319	0.66	444	**0.72**	**0.75**	1.48
$FPDME_8$(7,1,4,no trunc.) [29]	956	0.83	1521	3.89	2.83	0.77
$FPDME_8$(7,1,4,nt=16) [29]	421	0.72	599	1.15	1.08	0.81
FPAD_L4A2 [30]	368	0.66	429	**0.71**	**0.87**	4.61
FPAD_L4A3 [30]	601	0.81	689	1.68	1.74	2.88

For example, PLSAD(6) achieves lower EDP and MRED compared to all variants of LPCAD, FPDME, and FPAD. This trend is also observed for ADP, with the exception of FPDME4(7,1,3,nt = 17). When considering both error and overall performance metrics, these results position the proposed design as competitive with state-of-the-art dividers. Because the EF Extract Unit and Shift Unit are not included in, and the output bit-widths (related to the number of output buffers) are different, the proposed floating-point dividers have relatively much less area, delay and power than the proposed fixed-point designs in Section II.

TABLE VIII
THE PSNR/SSIMS OF THE COLOR QUANTIZATION RESULTS USING PLSAD WITH DIFFERENT M

m	8	7	6	5
Lena	32.8dB/0.99	31.8dB/0.98	30.6dB/0.98	24.0dB/0.94
Peppers	36.9dB/0.99	36.6dB/0.99	33.8dB/0.99	25.4dB/0.94
Kodim	33.0dB/0.96	29.6dB/0.93	28.4dB/0.91	23.2dB/0.87

IV. EXPERIMENTAL APPLICATIONS AND CASE STUDY

This section is dedicated to examining the disparities between the outcomes of the proposed approximate divider and the exact divider in practical applications. We conduct several benchmark tests, include image processing approaches such as change detection, background removal, K-Mean image color quantization, and Softmax layers in neural networks.

A. Change Detection

Change detection is a widely used technique to identify differences between two greyscale images. This process involves dividing the pixel values of two grey-scale images to obtain an output image that displays only the dissimilarities between them. If there are differences, the resulting pixels in the area of intensity change deviates in value from 1, which indicates the level of difference between the two input images.

This test used unsigned 8-by-8 PLSAD to verify the applicability of the approximate divider in processing two 8-bit greyscale images. Meanwhile, unsigned 8-by-8 AXDr3_TR(10), TruncApp_AM(5) and EAXHD(8) were compared for image quality. The quotient obtained from this process was then stretched to a range of [0,255] to restore a normal display of the image in greyscale. Fig. 13 presents the two input images alongside the peak signal-to-noise ratio (PSNR) and structural similarity (SSIM) values of four output images processed by both the exact divider and the PLSAD with m = 8,6,4. Results indicate that the division effect with m = 8 and 6 is almost identical to that of the exact divider. Meanwhile, the division effect with m = 4 remains clear.

B. Background Removal

Background removal is a common pre-processing step in image analysis that eliminates irrelevant background elements to improve the visibility of the target object. In high-resolution images processing, an approximate divider can be utilized instead of exact division for enhanced computational efficiency without much loss of accuracy. The implementation process involves dividing two pixels with identical coordinates in two pictures. A higher accuracy is required in background removal, so we employ an unsigned 16-by-16 PLSAD with different *m* values and modern dividers to evaluate PSNR and SSIM in this test.

Fig. 14 illustrates the outcomes of employing PLSAD with m values of 8, 6, and 5. It is visually evident that the PLSAD(6) method still exhibits comparable results to the exact divider. Hence, the selection of m-values can be based on the desired PSNR threshold and available hardware resources.

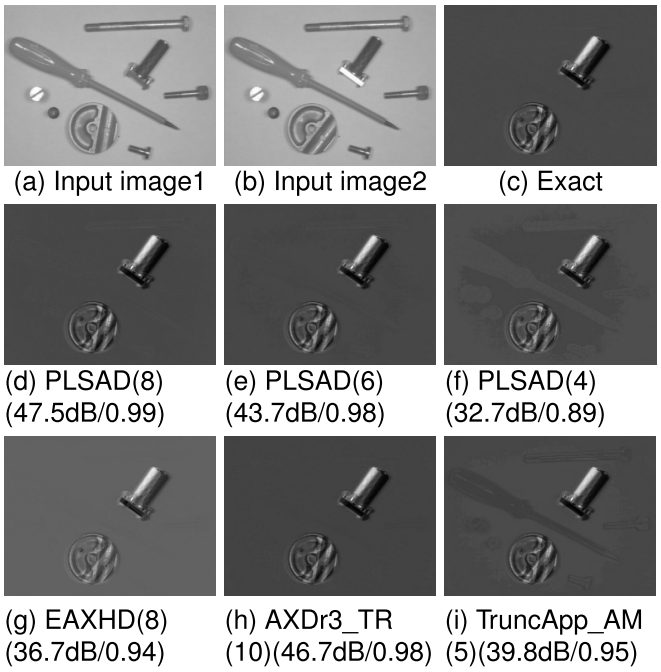

Fig. 13. The PSNR/SSIM of change-detection using exact divider, PLSAD with m = 8, 6, 4 and other dividers.

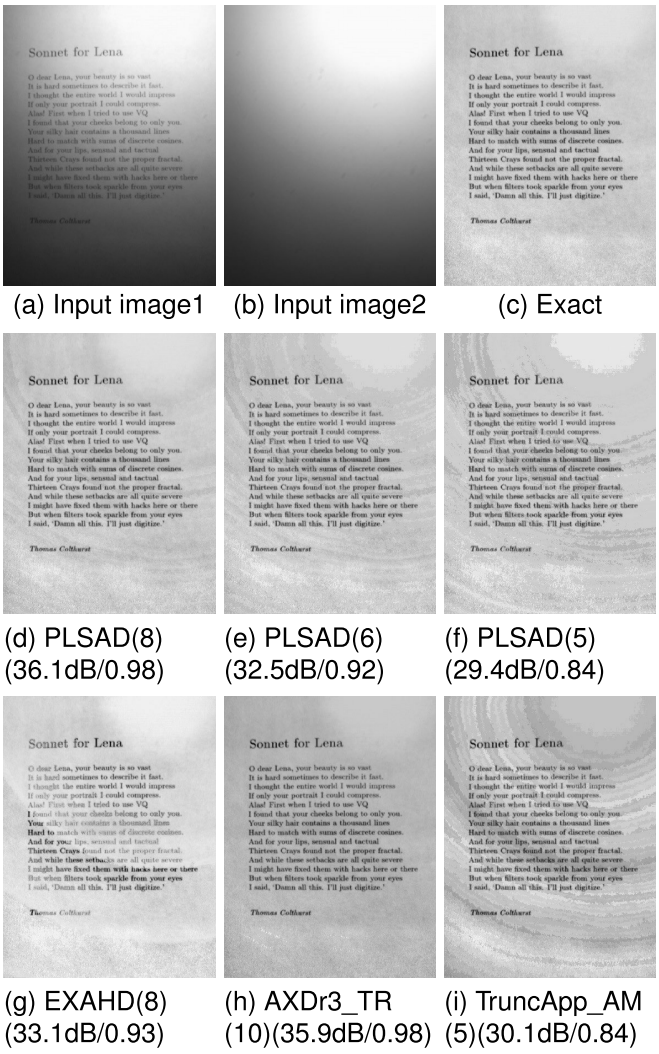

Fig. 14. Background removal using exact divider, PLSAD with m = 8, 6, 5 and other approximate dividers.

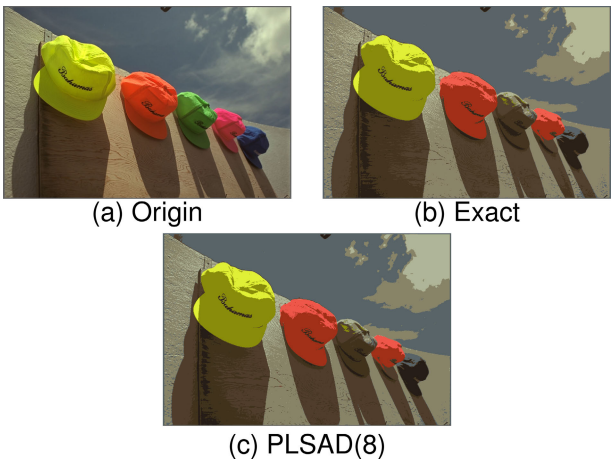

Fig. 15. Color quantization of 'Kodim' using Exact and PLSAD(8).

C. K-Means Color Quantization

Color quantization is another typical technique in digital image processing to reduce the number of colors in an image and obtain a compressed image that resembles the original. The key problem in quantization is to find a suitable color palette by first selecting several random colors in the image color space as an initial palette. The K-Means algorithm is a prominent method employed for color quantization, wherein each pixel value in the image is divided into K classes and then mapped to the initial palette based on their proximity to the color pixel values. This generates K clustering centres, and the average of each class's pixel values is calculated by dividing the sum by the number of pixels, which yields a new palette. This iterative process is repeated until the colors in the palette converge to a stable state.

The K-Means color quantization results are presented in Table VI, wherein image 'Lena' and 'Peppers' were quantized into six different colors, while image 'kodim' was quantized into ten different colors. After constant iterative division, it revealed that PLSAD with m = 8, 7, 6 exhibited satisfactory quality.

D. Softmax Layer

The softmax function is usually placed as a normalisation function in the last layer of a neural network to deal with multi-classification problems. It converts the multiple outputs of the fully connected layers into a probability distribution to predict the classification outcome of the neural network model. It's expression is shown in Eq. 18. P is the predicted probability of the j^{th} class in the sample vector x. The weight vector W contains the weights of each class, and K denotes the total number of classes. To avoid numerical overflow during computation, the maximum value of $e^{x^T W_k}, k = 1 \ldots K$, is subtracted by c.

$$P = \frac{e^{x^T W_j}}{\sum_1^K e^{x^T W_k}} = \frac{e^{x^T W_j - c}}{\sum_1^K e^{x^T W_k - c}} \tag{18}$$

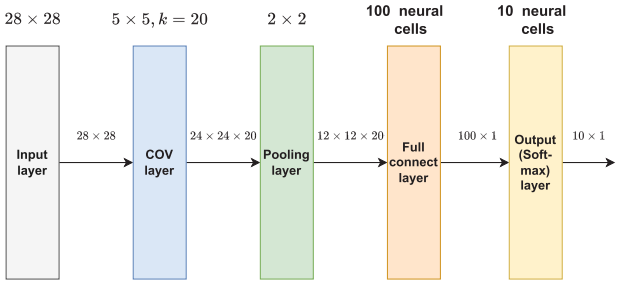

Fig. 16. The structure of Lenet adopted in this paper.

TABLE IX
ACCURACY OF PLSAD WITH DIFFERENT M VALUES FOR SOFTMAX LAYER

	Exact	PLSAD(10)	PLSAD(8)	PLSAD(6)	PLSAD(4)
accuracy	95.8%	95.8%	95.8%	95.65%	94.2%

In this test, a classical LeNet architecture is developed, which is composed of Convolutional Layer, Pooling Layer, Activation Function, Full Connection Layer, and Softmax Output Layer. The neural network model comprises four layers, as depicted in Fig. 16, with each input image having dimensions of 28×28 pixels. The convolutional layer contains 20 features with a kernel size of 9×9, and pooling operations are performed with a size of 2×2. A fully connected layer comprising 100 neurons precedes the Softmax Layer that outputs probabilities for ten distinct classes. We use the MINST handwritten digit database [31] as our training dataset, which contains ten sample labels. To validate the practicality and reliability of our proposed design, we replace the exact divider in the softmax layer with an approximate divider to evaluate the probability of correct classification when applying test data.

Upon conducting validation tests with a dataset comprised of 10000 MNIST test images, the results of LeNet exhibited remarkably accurate classifications, with the average probability being nearly identical to that achieved with the exact division. Consequently, there is minimal variance between the network structures that utilize the approximate divider versus the exact division. Our results are shown in Table IX, which depicts the average probabilities of correct classification with the exact divider being 95.8%, while the corresponding values for different m using the approximate divider are 95.8%, 95.8%, 95.65%, and 94.2%. In essence, these figures reveal that no more than a 2% decrease in classification accuracy was observed. Thus, it is feasible to select PLSAD with varying m values to achieve an optimal balance between hardware performance and accuracy, depending on the specific demands of the neural networks.

E. Real-Time EEG Signal Processing

EEG signals hold promise for real-time classification of emotional states. These signals are characterized by the presence of distinct frequency bands that offer valuable information for emotion recognition. These frequency bands are: $\delta(< 4\ Hz), \theta(4-8\ Hz), \alpha(8-13\ Hz), \beta(13-30\ Hz)$, and $\gamma(> 30\ Hz)$, each of which has a specific scalp distribution and biological significance of rhythmic activity [32]. The DEAP dataset only stores EEG signal information in the 4-45 Hz frequency range to facilitate emotional classification after preprocessing, so that the beta band information can be ignored [33]. Among the 32 electrodes provided in the DEAP dataset for recording EEG signals, 4 of them was used, Fp_1, Fp_2, F_3 and C_4. Each electrode recorded the changes in the EEG signal of that subject during the viewing of a 1-minute music video at a sampling frequency of 512Hz. The original EEG signal was first downsampled to 128Hz for each channel separately. Next, the EEG signal was subjected to discrete wavelet transform(DWT) to extract its approximation and detail components, and we selected the 4th order decomposition order according to the five EEG frequency bands. Finally, the EEG signal was decomposed into four detail coefficients D_4, D_3, D_2 and D_1 and an approximation coefficients. After calculating, we obtain the energy of the approximation and detail components of each EEG signal segment and normalize the process to avoid excessive values. The energies of the detail and approximation components of the EEG signal are given by

$$ED_j = \sum_k \left|D_{j,k}\right|^2, \quad j = 1, 2, 3, 4 \tag{19}$$

$$EA_4 = \sum_k \left|A_{4,k}\right|^2 \tag{20}$$

where j is the wavelet decomposition level, ED_j is the energy of the detail component, and EA_4 is the energy of the approximation component obtained from the 4th level decomposition energy.

Since the energy of the EEG signal is concentrated within the frequency band of 0-64 Hz, the total energy is defined as

$$E_{total} = \sum_{j=1}^{4} ED_j + EA_4 \tag{21}$$

According to Eq. 20-22, we calculated the sum of squares for each component separately to extract the energy of each frequency band of the EEG signal, and then sumed the energy of each frequency band to obtain the total energy of the EEG signal. Subsequently, four dividers were used to obtain four eigenvalues as

$$\begin{cases} \gamma\% = ED_1/E_{total} \\ \beta\% = ED_2/E_{total} \\ \alpha\% = ED_3/E_{total} \\ \theta\% = ED_4/E_{total} \end{cases} \tag{22}$$

Here we replace the exact divider(based on 32-bit floating point) with our signed 16-by-16 approximate divider(m = 8). Fig. 17 displays the complete computation framework for the featured calculation module.

Taking subject 1 from DEAD dataset as an example, in order to achieve real-time EEG signal analysis, the 1-minute EEG signal was segmented by 32s with an interval of 0.0625s. We finally obtained 17,960 normalised energy data for each frequency band of each channel, with 16 groups in total. Table X shows the MRED of the exact and approximate normalised energy for each frequency band of each channel, and most of them are less than 1%. Consequently, normalization yields almost identical discrepancies between approximate

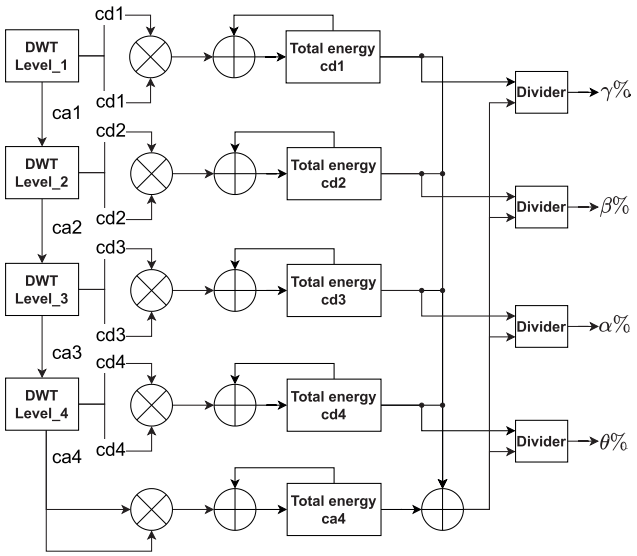

Fig. 17. The computation framework for the feature calculation module.

TABLE X
THE MRED OF NORMALISED ENERGY FOR EACH FREQUENCY BAND OF EACH CHANNEL

Channel	Frequency Band	MRED	Channel	Frequency Band	MRED
Fp_1	θ	0.82%	F_3	θ	0.82%
	α	0.87%		α	0.94%
	β	1.04%		β	1.11%
	γ	1.02%		γ	1.23%
Fp_2	θ	0.90%	C_4	θ	0.86%
	α	0.93%		α	0.89%
	β	0.70%		β	0.65%
	γ	1.16%		γ	0.79%
Total		0.92%			

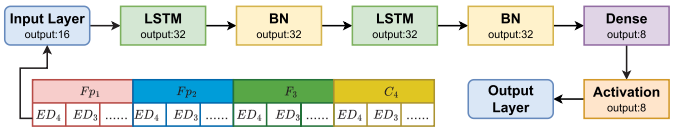

Fig. 18. The Structure of LSTM for Emotion Classification of EEG Signal.

and exact data, minimizing the impact on EEG signal classification.

A two-layer Long Short Term Memory (LSTM) network structure is shown in Fig. 18. The two-layer LSTM can analyse the inputs of energy features and extract more feature information. And batch Normalization layer (BN) normalizes the middle sample data. Dense layer further extracts the association between these features. The network can clssify 8 labels, so the output node of the last full-connection layer is identified to be 8, and a Sigmoid activation function is connected after each output node to complete the classification of the 8 kinds of emotions.

The validate accuracy for the emotion EEG classification using exact divider and approximate divider in feature calculation are 74.22% and 74.47%, respectively. It shows that our approximate method can retain almost all the energy information in the feature extraction process and its small error does not significantly affect the network classification results.

V. CONCLUSION

This paper has proposed approximate divider designs based on piecewise linear fitting of surface from new aspects. In contrast with conventional array dividers, the proposed design replaces the geometric "3D surface" of the mantissa dividing results with eight linear planes, accomplishing better experimental error metrics and hardware performance. To improve the designs in energy consumption and delay, we substitute the exact adder with an approximate OR-gate adder in the lower part computation. Therefore, the designs exhibit optimal balance in accuracy, hardware resources, and performance for low-power and energy efficient division. Notably, the proposed designs indicate high computational quality in both image processing, neural networks and real-time EEG signal processing applications with few errors. Moreover, it can be easily configured to trade-off among accuracy, power and hardware complexity according to application requirements.

REFERENCES

[1] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proc. IEEE*, vol. 108, no. 12, pp. 2108–2135, Dec. 2020.

[2] B. Liu et al., "A low power DNN-based speech recognition processor with precision recoverable approximate computing," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2022, pp. 2102–2106.

[3] V. Joshi, P. Mane, and C. K. Ramesha, "Cognitive approximate adder design for image processing applications," in *Proc. Int. Conf. Recent Trends Electron. Commun. (ICRTEC)*, Feb. 2023, pp. 1–6, doi: 10.1109/ICRTEC56977.2023.10111918.

[4] S. Venkataramani, S. T. Chakradhar, K. Roy, and A. Raghunathan, "Approximate computing and the quest for computing efficiency," in *Proc. 52nd ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, Jul. 2015, pp. 1–6.

[5] Q. Xu, T. Mytkowicz, and N. S. Kim, "Approximate computing: A survey," *IEEE Design Test*, vol. 33, no. 1, pp. 8–22, Feb. 2016.

[6] V. Gupta, D. Mohapatra, S. P. Park, A. Raghunathan, and K. Roy, "IMPACT: IMPrecise adders for low-power approximate computing," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design*, Oct. 2011, pp. 409–414.

[7] Z. Yang, A. Jain, J. Liang, J. Han, and F. Lombardi, "Approximate XOR/XNOR-based adders for inexact computing," in *Proc. 13th IEEE Int. Conf. Nanotechnol.*, Aug. 2013, pp. 690–693.

[8] K. Yin Kyaw, W. Ling Goh, and K. Seng Yeo, "Low-power high-speed multiplier for error-tolerant application," in *Proc. IEEE Int. Conf. Electron Devices Solid-State Circuits (EDSSC)*, Dec. 2010, pp. 1–4.

[9] P. Kulkarni, P. Gupta, and M. Ercegovac, "Trading accuracy for power with an underdesigned multiplier architecture," in *Proc. 24th Internatioal Conf. VLSI Design*, Jan. 2011, pp. 346–351.

[10] S. Venkatachalam and S.-B. Ko, "Design of power and area efficient approximate multipliers," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 5, pp. 1782–1786, May 2017.

[11] L. Chen, J. Han, W. Liu, and F. Lombardi, "On the design of approximate restoring dividers for error-tolerant applications," *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2522–2533, Aug. 2016.

[12] L. Chen, J. Han, W. Liu, and F. Lombardi, "Design of approximate unsigned integer non-restoring divider for inexact computing," in *Proc. 25th Great Lakes Symp. VLSI*, May 2015, pp. 51–56.

[13] H. Jiang, L. Liu, F. Lombardi, and J. Han, "Adaptive approximation in arithmetic circuits: A low-power unsigned divider design," in *Proc. Design, Autom. Test Eur. Conf. Exhibition*, Mar. 2018, pp. 1411–1416.

[14] R. Zendegani, M. Kamal, A. Fayyazi, A. Afzali-Kusha, S. Safari, and M. Pedram, "SEERAD: A high speed yet energy-efficient rounding-based approximate divider," in *Proc. Design, Autom. Test Eur. Conf. Exhibition*, Mar. 2016, pp. 1481–1484.

[15] S. Vahdat, M. Kamal, A. Afzali-Kusha, M. Pedram, and Z. Navabi, "TruncApp: A truncation-based approximate divider for energy efficient DSP applications," in *Proc. Design, Autom. Test Eur. Conf. Exhibition*, Mar. 2017, pp. 1635–1638, doi: 10.23919/DATE.2017.7927254.

[16] J. N. Mitchell, "Computer multiplication and division using binary logarithms," *IRE Trans. Electron. Comput.*, vols. EC-11, no. 4, pp. 512–517, Aug. 1962, doi: 10.1109/TEC.1962.5219391.
[17] W. Liu, T. Xu, J. Li, C. Wang, P. Montuschi, and F. Lombardi, "Design of unsigned approximate hybrid dividers based on restoring array and logarithmic dividers," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 1, pp. 339–350, Jan. 2022.
[18] Y. Wu et al., "An energy-efficient approximate divider based on logarithmic conversion and piecewise constant approximation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 7, pp. 2655–2668, Jul. 2022, doi: 10.1109/TCSI.2022.3167894.
[19] S. Gandhi, M. S. Ansari, B. F. Cockburn, and J. Han, "Approximate leading one detector design for a hardware-efficient Mitchell multiplier," in *Proc. IEEE Can. Conf. Electr. Comput. Eng. (CCECE)*, May 2019, pp. 1–4.
[20] Z. Babić, A. Avramović, and P. Bulić, "An iterative logarithmic multiplier," *Microprocessors Microsystems*, vol. 35, no. 1, pp. 23–33, Feb. 2011.
[21] K. Kunaraj and R. Seshasayanan, "Leading one detectors and leading one position detectors–An evolutionary design methodology," *Can. J. Electr. Comput. Eng.*, vol. 36, no. 3, pp. 103–110, Summer. 2013, doi: 10.1109/CJECE.2013.6704691.
[22] S. Veeramachaneni, K. Krishna, L. Avinash, S. Puppala, and M. B. Srinivas, "Novel architectures for high-speed and low-power 3–2, 4–2 and 5–2 compressors," in *Proc. 20th Int. Conf. VLSI Design Held Jointly 6th Int. Conf. Embedded Syst. (VLSID)*, 2007, pp. 324–329.
[23] H. R. Mahdiani, A. Ahmadi, S. M. Fakhraie, and C. Lucas, "Bio-inspired imprecise computational blocks for efficient VLSI implementation of soft-computing applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 57, no. 4, pp. 850–862, Apr. 2010.
[24] (2011). *Nangate 45 Nm Open Cell Library*. [Online]. Available: http://www.nangate.com/
[25] S. Venkatachalam, E. Adams, and S.-B. Ko, "Design of approximate restoring dividers," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2019, pp. 1–5.
[26] E. Adams, S. Venkatachalam, and S.-B. Ko, "Approximate restoring dividers using inexact cells and estimation from partial remainders," *IEEE Trans. Comput.*, vol. 69, no. 4, pp. 468–474, Apr. 2020.
[27] J. Melchert, S. Behroozi, J. Li, and Y. Kim, "SAADI-EC: A quality-configurable approximate divider for energy efficiency," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 11, pp. 2680–2692, Nov. 2019.
[28] O. G. Ratnaparkhi and M. Rao, "LEAD: Logarithmic exponent approximate divider for image quantization application," in *Proc. Great Lakes Symp. VLSI*, Jun. 2022, pp. 1–20.
[29] G. Di Meo, A. G. M. Strollo, and D. De Caro, "Novel low-power floating-point divider with linear approximation and minimum mean relative error," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 5275–5288, Dec. 2023, doi: 10.1109/tcsi.2023.3312974.
[30] C. K. Jha, K. Prasad, V. K. Srivastava, and J. Mekie, "FPAD: A multi-stage approximation methodology for designing floating point approximate dividers," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Seville, Spain, Oct. 2020, pp. 1–5, doi: 10.1109/ISCAS45731.2020.9180768.
[31] Y. LeCun, C. Cortes, and C. Burges. (2019). *The MNIST Database of Handwrittendigits*. [Online]. Available: http://yann.lecun.com/exdb/mnist/
[32] G. Chen, X. Zhang, Y. Sun, and J. Zhang, "Emotion feature analysis and recognition based on reconstructed EEG sources," *IEEE Access*, vol. 8, pp. 11907–11916, 2020.
[33] M. Khateeb, S. M. Anwar, and M. Alnowami, "Multi-domain feature fusion for emotion classification using DEAP dataset," *IEEE Access*, vol. 9, pp. 12134–12142, 2021.

Chaoyuan Wu received the B.S. degree from Guangdong University of Technology, China, in 2021. He is currently pursuing the M.E. degree in integrated circuit design with Shenzhen University, China. His current research interests include approximate computing circuits and brain–computer interface chips.

Weiwei Shi (Member, IEEE) received the B.S. degree in electronic engineering and the M.E. degree in microelectronics and solid-state electronics from South China University of Technology in 2004 and 2007, respectively, and the Ph.D. degree in electronic engineering from The Chinese University of Hong Kong (CUHK) in 2011. He is currently an Associate Professor with the Department of Microelectronics, Shenzhen University, and also a member of the State Key Laboratory of Radio Frequency Heterogeneous Integration (Shenzhen University). His current research interests include IC designs on power-performance-area efficient AI chips, approximate computation, subthreshold digital systems, and human–computer interface chips.

Yida Yuan received the B.S. degree in electrical engineering from The Pennsylvania State University in 2019 and the M.S. degree in electrical engineering from the University of Southern California in 2022. He is currently with WingSemi Technology (Shanghai), with a focus on RISC-V architecture and customized processor design.

Zhuoliang Zou received the B.S. degree in microelectronic science and engineering from Shenzhen University, Shenzhen, China, in 2023. He is currently pursuing the M.E. degree in electronic science and technology with ShanghaiTech University, China. His current research interests include silicon optical devices and photonic integrated circuit.

Zhihong Mo received the B.S. degree from Dongguan University of Technology, China, in 2021. He is currently pursuing the M.E. degree in integrated circuit design with Shenzhen University, China. His current research interests include approximate computing circuits and brain–computer interface chips.

Jiangwei He received the B.S. degree from Zhengzhou University, China, in 2021. He is currently pursuing the M.E. degree in integrated circuit design with Shenzhen University, China. His current research interests include RISC-V and neural network accelerator.